THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 01

AI Design #73 — Unified Enterprise Agentic Platform

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS
READ · REFLECT · PRACTICE

01

YOUR STUDY COMPANION

Enterprise IT Operations Assistant — Stages 1–4 and Stage 5, Evolution 1

Case: Fictional Checkout API incident `INC-42`

Learning approach: Problem → Concepts → Minimal architecture → Local mock → Production evolution

Coverage: Stage 1, Stage 2, Stage 3, Stage 4, and **Stage 5 Evolution 1 (bounded model-driven tool calling)**. Stage 5 Evolution 2 and Stage 6 are **not covered**.

Data: All incident records, identities, metrics, and deployment records in this guide are **synthetic**.

> **Safety rule:** AI-generated explanations and tool requests are not authorization. The application and business systems enforce permissions. No part of the implemented baseline can modify production systems.

What we have built so far

We are **not designing the entire enterprise platform in one step**. Each stage adds only the capabilities justified by INC-42. The platform-wide concerns remain future work.

- **Stage 1** — Problem, actors, priorities and trust boundaries
- **Stage 2** — LLMs, workflows, tool calling and grounding
- **Stage 3** — Two-tool read-only system and typed contracts
- **Stage 4** — Runnable synthetic Python workflow and tests
- **Stage 5A** — Bounded model tool selection and enforcement

> **Central lesson** The LLM may explain evidence and propose what to inspect next. **Application

code and business systems** decide what data can be read and what actions can

> run. No production write action exists in our current design.

An enterprise runs an e-commerce platform. Customers use its **Checkout API** to complete purchases.

NAVIGATION

Inside this guide

01 1.1 Business problem

02 1.2 Representative user request

03 1.3 Actors

04 1.4 Functional requirements

05 1.5 Ranked non-functional requirements

06 1.6 Sources of truth

07 1.7 Trust boundaries

08 1.8 First-version scope

09 Next — Stage 2: Essential Concepts

CHAPTER

1.1 Business problem

Customers experience slow checkouts, and the IT Operations team receives a severity-2 alert.

Monitoring reports a p95 latency rise from **240 ms to 1,800 ms**, while error rate rises from **0.4% to 3.1%**. Deployment `v2.7.4` occurred at 09:55, shortly before the degradation.

Fictional incident: INC-42 Checkout API is slow [SEV-2]

Signal	Normal	During incident
p95 latency	240 ms	1,800 ms
Error rate	0.4%	3.1%
Deployment	—	`v2.7.4` at 09:55

Illustrative observations:

- Latency is near normal around 09:45.
- Deployment `v2.7.4` occurs at 09:55.
- Latency increases sharply afterward, reaching 1,800 ms around 10:10.
- **Temporal correlation does not prove the deployment caused the slowdown.**

> **Do not confuse correlation with cause** A deployment near the incident is a lead for investigation, not proof that the deployment caused the slowdown.

The on-call engineer needs to answer:

- What changed?
- What evidence explains the slowdown?
- What is the most likely cause?
- What should we investigate or do next?
- Can we safely recover the service?

The on-call engineer currently checks monitoring dashboards, logs, deployments, incident records, and runbooks separately.

The process is slow, repetitive and vulnerable to human error under pressure.

Business objective

Help IT Operations engineers **investigate incidents faster**, **understand evidence, and recommend safe next steps**, while retaining human responsibility for remediation.

Desired outcomes include:

- lower investigation effort,
- potentially shorter mean time to resolution (MTTR),
- fewer unsafe operational decisions.

These are **goals, not measured improvements**.

Current architectural limit

One bounded incident investigator, two read-only tools, synthetic records, an optional model adapter, and no enterprise-scale platform services.

CHAPTER

1.2 Representative user request

> Investigate incident INC-42. Checkout API latency increased. Find possible causes and recommend the safest next step.

1.2.1 Representative user journey

Our representative request:

> "Investigate incident INC-42. Checkout API latency increased. Find possible causes and recommend the safest next step."

1. **Open the incident**. The engineer selects `INC-42` and begins the investigation.
2. **Verify access**. The application checks the engineer's identity and permissions before retrieving protected information.
3. **Collect evidence**. The assistant obtains latency measurements, error rates and recent deployment information from approved systems.
4. **Analyze findings**. The assistant compares the incident timeline with recent changes and identifies possible explanations.
5. **Present conclusions**. It separates confirmed observations from hypotheses, identifies evidence sources and reports missing information.
6. **Attach evidence**. The assistant links relevant data sources to the findings.
7. **Recommend next steps**. The assistant does not independently change production.
8. **Engineer reviews**. The engineer reads the report and decides whether to take action.

Example of the expected response

```
**Incident investigation — INC-42**                          [Needs review]

**Observed facts**

- Checkout API p95 latency increased from 240 ms to 1,800 ms.
  Evidence: `METRIC-07`.

- Deployment v2.7.4 occurred shortly before degradation.
  Evidence: `CHANGE-03`.

**Hypothesis**
The recent deployment may be contributing to the slowdown.
Available evidence does not establish the root cause.

**Recommended next step**
Compare request latency and database connection metrics before and after
deployment.

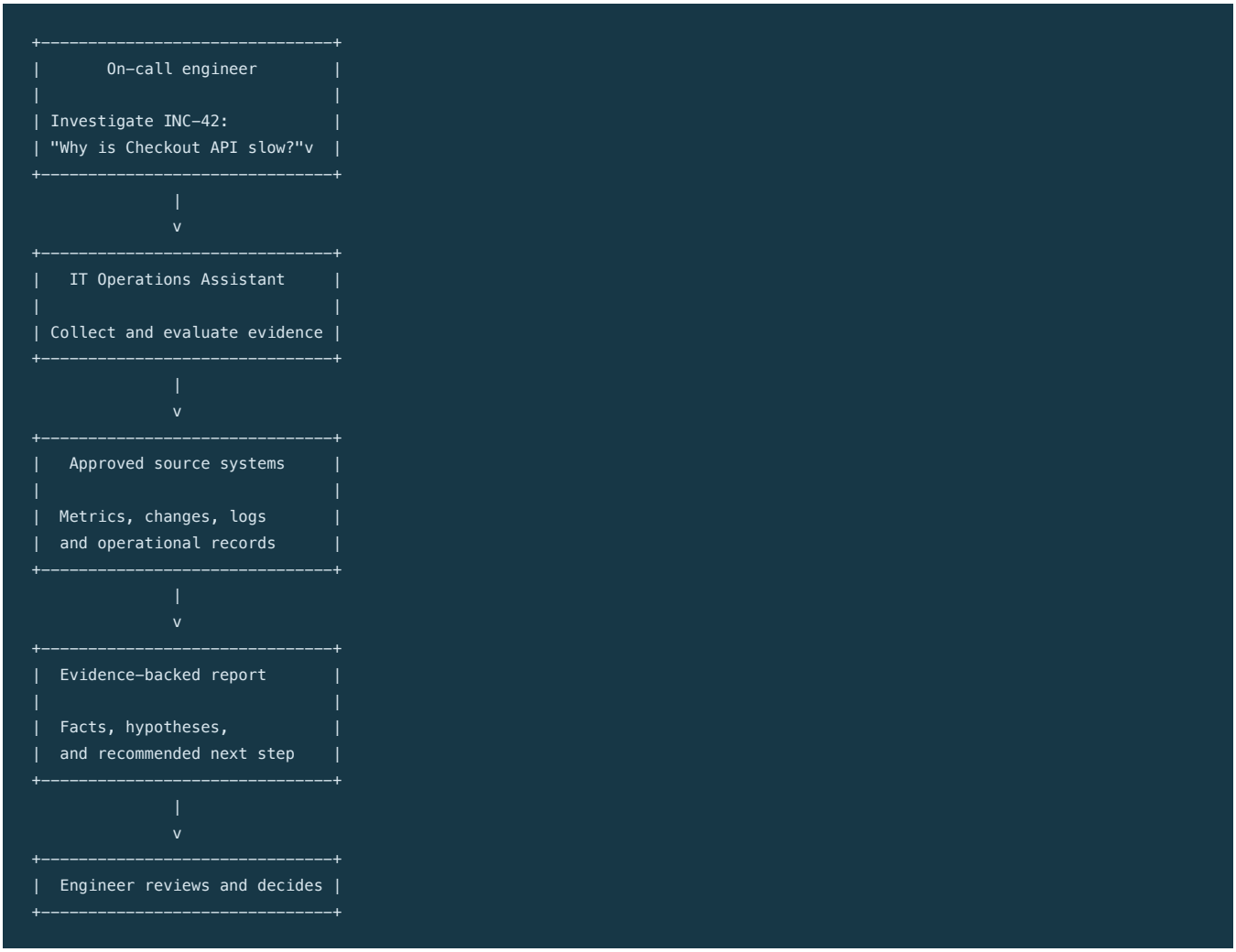
If findings support a rollback, ask the authorized operator to review that
option.

[No production action executed. Synthetic example output.]
```

Notice that the result contains observations, a hypothesis and a recommendation. These are three different kinds of information.

A well-designed assistant must not turn a weak hypothesis into a confirmed root cause.

Business-level sequence



This is a **business flow**, not yet a deployment architecture.

CHAPTER

1.3 Actors

Actor	Main responsibility	Authority boundary
On-call engineer	Starts investigations and reviews findings	May read only permitted operational evidence
Incident commander	Coordinates response and incident status	Manages incident within assigned role
SRE / service owner	Checks technical findings and remediation options	Can operate owned services under policy
Change approver	Reviews proposed production changes	Approves only eligible actions
Security / compliance team	Defines identity, policy, and audit rules	Governs identity, policy and security controls

Shared platform team	Owns reusable AI platform capabilities later	Does not own every service's business semantics

Primary first-version user: The on-call engineer.

CHAPTER

1.4 Functional requirements

Functional requirements describe what the system must do.

Priority definitions:

- **P0 — Must have:** Necessary for a useful, safe first version.
- **P1 — Should have:** Important for a stronger operational experience.
- **P2 — Later:** Deferred until requirements justify the complexity.

ID	Requirement	Priority	Architectural significance	Architectural consequence
F1	Accept an incident ID, service, and question	P0	Establishes a clear investigation target	Requires an investigation entry point
F2	Retrieve authorized metrics and deployment data	P0	Requires reliable access to existing systems	Requires approved read-only integrations
F3	Explain observations and possible causes using evidence	P0	Makes source provenance necessary	Requires provenance and report structure
F4	Recommend safe next steps without executing remediation	P0	Separates AI recommendations from business actions	Separates recommendation from action
F5	Enforce user-specific data access	P0	Requires authorization outside the model	Requires server-side authorization
F6	Handle missing, stale, or conflicting evidence	P1	Requires an explicit uncertainty and failure path	Requires incomplete/uncertain result states
F7	Present an incident timeline and readable report	P1	Shapes the future user experience	Requires useful presentation and event data
F8	Reuse tools across teams and applications	P2	May later justify shared platform interfaces	May later justify a shared protocol/catalog
F9	Support human-approved remediation	P2	May later require durable state and action auditing	May later require durable workflow state

F9 is intentionally deferred. The first version is read-only.

CHAPTER

1.5 Ranked non-functional requirements

Rank	Constraint	First-version rule	Why
1	Identity and authorization	No unauthorized incident access	Operational evidence can be sensitive or tenant-scoped.
2	Action safety	Zero production writes	A wrong action can worsen an outage
3	Evidence integrity	Material findings cite available evidence	A plausible answer without valid evidence is not reliable.

4	Reliability and responsiveness	Explicit tool failures and usable partial results	Engineers need honest responses during incidents
5	Auditability and cost	Track investigation steps and bound work	Operators need traceable, predictable execution.

Illustrative future latency target: An initial investigation summary within 30 seconds for 95% of requests. This was a proposed design target, **not an agreed or measured SLO**.

A faster response does not justify skipping authorization or inventing unavailable facts.

CHAPTER

1.6 Sources of truth

Information	Authoritative owner	Assistant's permitted role
Latency and error measurements	Monitoring system	Read and explain authorized records
Application logs	Logging system	Read recent changes (permitted records); do not infer causality automatically
Deployments	Deployment/change system	Read versions and timestamps
Incident metadata	Incident management system	Read permitted incident state; never silently rewrite it
Runbooks	Approved runbook repository	Retrieve guidance when later added
Identity and permissions	Enterprise identity and business policy systems	Use independently verified decisions; do not invent grants.
Remediation execution	Authorized operations service and human process	Recommend only in the initial scope.

The model's answer is **not** a source of operational truth.

CHAPTER

1.7 Trust boundaries

1. A chat request does not grant access to all services.
2. The API must derive identity and permissions from verified server-side context.
3. Each data source must enforce its own authorization rules.
4. Tool responses, logs, documents, and model-generated text are **untrusted data**, not executable instructions.
5. A suggested rollback or an approval button is **not** authorization to perform a rollback.

CHAPTER

1.8 First-version scope

Included:

- One fictional enterprise and IT Operations team.
- One incident: `INC-42`, Checkout API.
- One authorized on-call engineer.
- Two read-only capabilities: retrieve metrics and deployment history.
- Evidence-backed findings, uncertainty, and recommendations.
- Synthetic source data only.

Excluded initially:

- Automatic restart, rollback, or infrastructure writes.
- Real credentials or production integrations.
- Multi-agent delegation, MCP, A2A, RAG, and advanced UI protocols.
- Global multi-region platform infrastructure.

Explicit assumptions

Working assumptions: The incident has a known tenant and service; the authorized operator may read its records; two sources are enough for the first demonstration; the application has no write permission.

Assumption	Reason
All incident data is synthetic	Safe, reproducible learning
One authorized engineer can access the selected evidence	Keeps the first authorization boundary small
Metrics and deployment history are available through read-only interfaces	Supports the first investigation workflow
The assistant has no production write permission	Prevents unintended side effects
Deployment timing alone cannot confirm root cause	Forces evidence-based reasoning

Stage 1 result

We defined an **investigation copilot**, not an autonomous operations controller. Safety, provenance, and ownership take priority over sophistication.

CHAPTER

Next — Stage 2: Essential Concepts

Stage 2 will explain the minimum concepts needed to turn our requirements into a working assistant:

- What an LLM, an AI agent and tool calling mean in this incident.
- How the assistant uses evidence from real source systems.
- Why deterministic workflows may be simpler than autonomous agent behavior.
- How to separate model decisions from authorized operations.

We will use the same INC-42 incident throughout, without jumping to the full production architecture.

Continue to Stage 2